Tabăra de pregătire a lotului național de informatică juniori

Ploiești, 23-28 aprilie 2026

Descrierea soluțiilor Baraj 1

Comisia științifică

25 aprilie 2026

Problema 1. Armonioase

Propusă de: șef lucrări dr. Mihai Nan, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București

Vom folosi următoarele notații:

- W = numărul de litere w, M = numărul de litere m;
- VV = numărul de secvențe disjuncte vv, NN = numărul de secvențe disjuncte nn.

Atunci:

$$\text{numărul final de } w = W + x, \quad 0 \le x \le VV$$

$$\text{numărul final de } m = M + y, \quad 0 \le y \le NN$$

Un șir este armonios dacă: $(w \mid m) \iff (M + y) \equiv 0 \pmod{W + x}$.

Cerința 1

Pentru fiecare valoare K, trebuie verificat dacă există x, y astfel încât $M + y = K$, adică $y = K - M (0 \le y \le NN)$ și există x astfel încât: $(W + x) \mid K$.

Pentru a determina rezultatul, calculăm $y = K - M$, verificăm dacă $0 \le y \le NN$ și iterăm cu x în $0..VV$ pentru a verifica dacă $W + x > 0$ și $K \bmod (W + x) \equiv 0$. Dacă găsim un astfel de x, răspunsul este **DA**.

După ce determinăm valorile pentru W, M, VV, NN, complexitatea pentru fiecare interogare este $O(VV)$.

Cerința 2

Trebuie să numărăm perechile distincte (val_1, val_2) unde $val_1 = W + x$, $val_2 = M + y$ și $(W + x) \mid (M + y)$. Pentru fiecare x fix, notăm $d = W + x$. Problema se rezumă atunci la găsirea tuturor valorilor lui y astfel încât $M + y$ să fie multiplu de d, adică $M + y = k \cdot d$ pentru un număr natural k. Rescriind, obținem $y = k \cdot d - M$. Observăm că atunci când creștem k cu 1, valoarea lui y crește exact cu d. Aceasta înseamnă că valorile posibile ale lui y nu sunt alese la întâmplare, ci apar la distanță egală de d: considerăm y_0 prima valoare bună, apoi urmează $y_0 + d$, $y_0 + 2d$ și așa mai departe. O astfel de listă de numere, în care diferența dintre două valori consecutive este mereu aceeași, se numește **progresie aritmetică**.

Concret, trebuie mai întâi să găsim cel mai mic multiplu de d care este mai mare sau egal cu M; diferența dintre acest multiplu și M ne dă prima valoare validă y_0. După aceea, toate celelalte valori valide pentru y se obțin adăugând repetat d, adică vom avea șirul de valori y_0, $y_0 + d$, $y_0 + 2d, \ldots$. Ne oprim când depășim limita maximă permisă, adică NN. Astfel, pentru fiecare x, valorile bune ale lui y formează o progresie aritmetică cu rația $d = W + x$, iar noi trebuie doar să numărăm câte dintre aceste valori se află în intervalul $[0, NN]$.

Determinăm primul y valid $startY = \left\lceil \frac{M}{W+x} \right\rceil \cdot (W + x) - M$.

Dacă $0 \leq startY \leq NN$, atunci numărul de soluții este: $\left\lfloor \frac{NN - startY}{W+x} \right\rfloor + 1$. Iterăm pentru toate valorile $x \in \{0, 1, \ldots VV\}$.

Cerința 3

Trebuie determinată lungimea maximă a unei secvențe ce este armonioasă. Nu putem aplica nicio modificare asupra șirului inițial. Astfel, observăm că doar literele w și m contează, pentru că acestea determină dacă o secvență este armonioasă sau nu. Celelalte litere nu influențează condiția de armonie.

Compresia șirului

Construim un vector care reține doar pozițiile literelor `w` și `m`. Fie $a_1, a_2, \ldots, a_T$ literele relevante, iar $pos_1, pos_2, \ldots, pos_T$ pozițiile lor în șirul inițial.

Adăugăm și valori pentru extremități: $pos_0 = -1, pos_{T+1} = lg$; unde lg este lungimea șirului.

Prefixe

Definim prefixele:

$$prefix_w[k] = \text{numărul de litere } w \text{ în } a_1, a_2, \ldots, a_k,$$

$$prefix_m[k] = \text{numărul de litere } m \text{ în } a_1, a_2, \ldots, a_k.$$

Pentru secvența de litere relevante cuprinsă între pozițiile $i + 1$ și j, numerele de apariții ale literelor w și m sunt:

$$\Delta w = prefix_w[j] - prefix_w[i], \qquad \Delta m = prefix_m[j] - prefix_m[i].$$

Secvența este armonioasă dacă: $\Delta w > 0, \qquad \Delta m > 0, \qquad \Delta m \equiv 0 \pmod{\Delta w}$.

Echivalent, există un număr natural $q \geq 1$ astfel încât: $\Delta m = q \cdot \Delta w$.

Vom împărți soluția în două cazuri, alegând $B = \lfloor \sqrt{T} \rfloor$.

Cazul 1: raport mic $q = \frac{\Delta m}{\Delta w} \leq B$

Fixăm o valoare q între 1 și B. Căutăm toate secvențele pentru care $\Delta m = q \cdot \Delta w$.

Rescriind cu prefixe:

$$prefix_m[j] - prefix_m[i-1] = q \cdot \big(prefix_w[j] - prefix_w[i-1]\big),$$

de unde obținem:

$$prefix_m[j] - q \cdot prefix_w[j] = prefix_m[i-1] - q \cdot prefix_w[i-1].$$

Așadar, pentru un q fix, o secvență este armonioasă atunci când este îndeplinită relația: $val[j] = val[i-1]$, unde $val[k] = prefix_m[k] - q \cdot prefix_w[k]$.

Aceasta înseamnă că dacă pentru două poziții diferite avem aceeași valoare pentru expresia $prefix_m[k] - q \cdot prefix_w[k]$, atunci secvența determinată de aceste două poziții este armonioasă.

Pentru fiecare valoare a lui q până la B:

- memorăm prima apariție a fiecărei valori $prefix_m[i] - q \cdot prefix_w[i]$, astfel încât să știm de unde începe o posibilă secvență armonioasă;
- dacă aceeași valoare reapare mai târziu, secvența dintre prima apariție și poziția curentă reprezintă o secvență validă, iar lungimea acesteia poate fi comparată cu lungimea maximă găsită până acum.

Observăm că valorile expresiei $val[k]$ sunt limitate într-un interval relativ mic.

Pentru orice poziție k, avem $0 \leq prefix_m[k] \leq T$ și $0 \leq prefix_w[k] \leq T$. Rezultă $-q \cdot T \leq val[k] \leq T$.

Cum în acest caz $q \leq B$ și alegem $B \approx \sqrt{T}$, valorile lui $val[k]$ sunt cuprinse într-un interval de lungime $O(T\sqrt{T})$. În practică, însă, toate valorile interesante pentru val, care să mai permită o a doua apariție pe o poziție viitoare, sînt cuprinse în intervalul $[-T, T]$.

Pentru a evita valorile negative, putem translata aceste valori adăugând o constantă (de exemplu, lg), astfel încât toate valorile să devină nenegative.

În consecință, putem folosi un vector de dimensiune $O(lg)$ (de exemplu $2 \cdot lg + 2$) pentru a reține prima apariție a fiecărei valori $val[k]$, fără a avea nevoie de o structură de tip map, care ar fi mult prea lentă.

Parcurgând toate valorile posibile ale lui q și actualizând constant lungimea maximă, putem afla în final lungimea maximă a unei secvențe armonioase în șirul inițial.

Cazul 2: raport mare $q = \frac{\Delta m}{\Delta w} > B$ **(număr mic de** `w`**)**

Dacă $q = \frac{\Delta m}{\Delta w} > B$, atunci: $\Delta m > B \cdot \Delta w$. Rezultă $\Delta m + \Delta w > (B+1)\Delta w$.

Dar $\Delta m + \Delta w$ este chiar numărul de litere relevante din secvență, deci:

$$\Delta w \leq \frac{T}{B+1}.$$

Cum alegem $B \approx \sqrt{T}$, obținem că în acest caz numărul de litere w din secvență este mic, adică de ordin cel mult $\sqrt{T}$.

Prin urmare, în loc să fixăm raportul q, fixăm $cntW = \Delta w$.

Pentru fiecare valoare $cntW = 1, 2, \ldots, B$, enumerăm toate grupurile de $cntW$ litere `w` consecutive în vectorul literelor relevante.

Dacă fixăm un astfel de grup, atunci orice secvență care conține **exact** aceste $cntW$ litere `w`:

- poate fi extinsă la stânga doar în blocul de litere `m` aflat imediat înaintea primei litere `w`;
- poate fi extinsă la dreapta doar în blocul de litere `m` aflat imediat după ultima literă `w`;
- nu poate trece peste următorul `w`, deoarece altfel numărul de litere `w` nu ar mai fi $cntW$.

Dacă notăm cu:

$$baseM = \text{numărul obligatoriu de litere } m \text{ dintre aceste } cntW \text{ litere } w,$$

iar cu u și v numărul de litere `m` pe care le mai adăugăm din blocul din stânga, respectiv din dreapta, atunci $\Delta m = baseM + u + v$.

Condiția de armonie devine $baseM + u + v \equiv 0 \pmod{cntW}$, iar pentru acest al doilea caz trebuie să mai avem și $\frac{\Delta m}{cntW} > B$.

În implementare, pentru fiecare început posibil și pentru fiecare $cntW$, se determină cel mai depărtat final valid al secvenței.

Fie $extra$ numărul de litere relevante pe care le mai putem adăuga după ultima literă `w` fixată. Trebuie să avem:

$$baseM + extra \equiv 0 \pmod{cntW}.$$

Alegem cel mai mic $extra$ care respectă această condiție, iar apoi îl mărim cât de mult posibil, din $cntW$ în $cntW$, fără să trecem peste următoarea literă `w`. Astfel, obținem cea mai lungă secvență armonioasă pentru alegerea curentă.

Dacă $baseM + extra = 0$, secvența nu conține nicio literă `m`, deci nu este considerată armonioasă. În acest caz, trecem la următoarea valoare posibilă a lui $extra$.

Cum $cntW \leq B$, iar pentru fiecare valoare procesăm pozițiile posibile de început, obținem o complexitate totală $O(T \cdot B)$.

În concluzie, împărțind problema în cele două cazuri:

- raport mic $\dfrac{\Delta m}{\Delta w} \leq B$;
- număr mic de litere w în secvență, corespunzător cazului $\dfrac{\Delta m}{\Delta w} > B$;

putem determina lungimea maximă a unei secvențe armonioase în $O(T\sqrt{T})$, unde T este numărul de litere relevante (`w` și `m`) din șir.

Cum $T \leq lg$, rezultă o complexitate finală $O(lg\sqrt{lg})$, suficient de bună pentru restricțiile din problemă.

Problema 2. Distrugere

Propusă de: prof. Petru Simion Oprița, Liceul Regina Maria Dorohoi

Subtask 1

Putem simula procesul descris în enunț testând fiecare element X ca fiind cel ales pentru distrugere. Pentru fiecare X, parcurgem întreg șirul și calculăm cel mai mare divizor comun dintre X și fiecare element $A[i]$ ($1 \leq i \leq N$). Numărăm câte elemente au $cmmdc(X, A[i]) > 1$ și reținem minimul acestor eliminări. Complexitatea acestei soluții este $O(N^2 \log V)$, unde V este valoarea maximă din șir.

Subtask 2

Putem optimiza soluția folosind un vector de frecvență pentru elementele șirului. În loc să parcurgem toate cele N elemente pentru fiecare X, vom parcurge doar valorile distincte din șir. Pentru un X fixat, numărul de elemente eliminate este $\sum_v \text{frecv}[v]$ pentru toate valorile v care nu sunt prime cu X. Sursele care implementează această optimizare trec testele din acest subtask.

Subtask 3

Pentru a obține punctajul maxim, trebuie să calculăm eficient câte elemente sunt eliminate de un X fără a parcurge restul valorilor. Un element $A[i]$ este eliminat dacă are cel puțin un factor prim comun cu X. Fie $\{p_1, p_2, \ldots p_k\}$ mulțimea factorilor primi distincți ai lui X. Numărul de elemente eliminate se calculează folosind Principiul Includerii și Excluderii:

$$|S_{p_1} \cup S_{p_2} \cup \cdots \cup S_{p_k}| = \Sigma|S_{p_i}| - \Sigma|S_{p_i} \cap S_{p_j}| + \ldots$$

unde S_d este mulțimea elementelor din șir divizibile cu d. Pentru implementare, urmăm pașii:

- Precalculare: Folosind ciurul lui Eratostene, factorizăm rapid orice număr în $O(\log V)$.
- Numărare divizori: Calculăm $cnt[d]$ reprezentând câte numere din șir sunt divizibile cu d, pentru toate valorile d până la 1.000.000. Aceasta se face în $O(V \log V)$ parcurgând multiplii fiecărui număr.
- Calcul PIE: Pentru fiecare valoare unică X din șir, generăm toate submulțimile factorilor săi primi (maximum $2^7 = 128$ combinații, deoarece produsul primelor 8 numere prime depășește 10^6). Pentru fiecare submulțime cu produsul d, adunăm sau scădem $cnt[d]$ din totalul elementelor eliminate.

Complexitatea finală este $O(V \log V + N \cdot 2^k)$, unde $k \leq 7$. Această soluție este optimă și se încadrează în limitele de timp și memorie.

Problema 3. Lalele

Propusă de: prof. Emanuela Cerchez, Colegiul Național „Emil Racoviță” Iași

Subtask 1

Pentru acest subtask este suficient să generăm recursiv toate soluțiile posibile și să le contorizăm. Vom reprezenta o soluție ca un vector *sol* în care reținem, în ordine crescătoare, indicii lalelelor care sunt culese. La fiecare pas vom reține într-un vector de frecvență *nr* numărul de lalele culese de fiecare culoare, precum și numărul de culori distincte în variabila *nrcd*.

```
void gen(int k)
//cand apelez gen(k) sunt ocupate in sol pozitiile 1, 2, ..., k-1
{int i;
 if (nrcd==K) //am obtinut o solutie
     nrsol++;
     for (i=sol[k-1]+1; i<=n; i++)
     //aleg floarea i pe pozitia k in buchet
     {
       if (nr[L[i]]>0) //daca culoarea florii i mai apare in buchet
          {nr[L[i]]++; sol[k]=i;
           gen(k+1);
           nr[L[i]]--;
          }
          else //culoarea acestei flori nu apare in buchet
          if (nrcd<K) //pot selecta inca o culoare?
             {nr[L[i]]=1; nrcd++; sol[k]=i;
              gen(k+1);
              nr[L[i]]=0; nrcd--;
             }
     }
}
```

Subtask 2

Generăm toate submulțimile formate din K culori dintre cele C existente și determinăm pentru fiecare submulțime în câte moduri putem forma un buchet care conține culorile din submulțimea respectivă. Vom reține o submulțime de culori în vectorul *sol*.

```
void combinari_iterativ()
//generez combinari de C luate cate K pentru a selecta culorile folosite
{int i;
 for (i=1; i<=K; i++) sol[i]=i;
 while (1)
       {nrsol+=numar_solutii();
        for (i=K; i>0 && sol[i]==c-K+i; i--);
        if (i==0) break;
            else
            {sol[i]++;
             for (i++; i<=K; i++) sol[i]=sol[i-1]+1;
            }
       }
}
```

Să notăm cu $nr[x]$=numărul de flori care au culoarea x. Pentru ca să apară culoarea x în buchet, putem alege orice mulțime de flori colorate în această culoare.
Există $2^{nr[x]} - 1$ astfel de mulțimi (scădem 1 pentru că nu vom considera și mulțimea vidă, trebuie să avem măcar o floare din culoarea respectivă).
Deci $2^{nr[sol[1]]} - 1$ posibilități pentru prima culoare, $2^{nr[sol[2]]} - 1$ posibilități pentru a doua culoare, $\ldots 2^{nr[sol[K]]} - 1$ posibilități pentru a K-a culoare.

În total, numărul de buchete care se pot forma folosind culorile din sol este $(2^{nr[sol[1]]} - 1) \cdot (2^{nr[sol[2]]} - 1) \cdot \dots \cdot (2^{nr[sol[K]]} - 1)$. Pentru a calcula $2^x - 1$ am utilizat operatori pe biți, dar la fel de simplu și eficient ar fi fost să precalculez puterile lui 2 într-un vector.

Funcția $numar_solutii()$ determină numărul de buchete ce se pot forma folosind culorile din sol

```
long long int numar_solutii()
{int i;
 long long int rez=1;
 for (i=1; i<=K; i++)
     rez=rez*((1LL<<nr[sol[i]])-1);
 return rez;
}
```

Ca optimizare minoră, care mai poate trece un test în plus, putem chiar să înlocuim fiecare valoare $nr[i]$ cu $2^{nr[i]} - 1$, deoarece aceasta este cantitatea care ne interesează și o vom accesa câte o dată pentru fiecare combinare care include culoarea i.

Subtask 3

Vom aborda problema prin programare dinamică.

Subproblemă:

Să se determine $nrsol[i][j]$ = numărul de posibilități de a obține exact i ($1 \leq i \leq K$) culori distincte din mulțimea de culori $1, 2, \dots j$ ($K \leq j$)

Pentru a obține i culori distincte selectate din mulțimea $1, 2, \dots j$ avem două variante:

1. nu folosim culoarea j, caz în care cele i culori le selectăm din mulțimea $1, 2, \dots j - 1$ (deci avem $nrsol[i][j - 1]$ posibilități);

2. folosim culoarea j; există $(2^{nr[j]} - 1)$ posibilități de a selecta o mulțime nevidă de flori de culoare j și pentru fiecare astfel de posibilitate există $nrsol[i - 1][j - 1]$ de a selecta $i - 1$ culori distincte din mulțimea $1, 2, \dots j - 1$; în total $nrsol[i - 1][j - 1] \cdot (2^{nr[j]} - 1)$ posibilități.

```
void pd()
{ int i, j;
  for (j=1; j<=c; j++)
       {for (i=1; i<=j; i++) nrsol[1][j]+=(1LL<<nr[i])-1;
        for (i=2; i<=min(K,j); i++)
            nrsol[i][j]=nrsol[i][j-1]+nrsol[i-1][j-1]*((1LL<<nr[j])-1);
       }
}
```

Subtask 4

Pentru a obține punctaj pentru acest subtask este necesară implementarea operațiilor cu numere mari.

Echipa

Problemele pentru această etapă au fost pregătite de:

- prof. Emanuela Cerchez, Colegiul Național „Emil Racoviță” Iași
- prof. Ionel-Vasile Piț-Rada, Colegiul Național Traian, Drobeta Turnu Severin
- șef lucrări dr. Mihai Nan, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Dorin-Mircea Rotar, Colegiul Național „Samuil Vulcan” Beiuș
- prof. Mihai Bunget, Colegiul Național „Tudor Vladimirescu” Târgu-Jiu
- prof. Ciprian Cheșcă, Liceul Tehnologic Grigore Moisil Buzău
- prof. Gheorghe-Eugen Nodea, Centrul Județean de Excelență Gorj
- prof. Petru Simion Oprița, Liceul Regina Maria Dorohoi
- prof. Zoltan Szabo, Inspectoratul Școlar Județean Mureș
- prof. Marius Nicoli, Colegiul Național Frații Buzești Craiova
- prof. Dana Lica, Centrul Județean de Excelență Prahova
- instr. Cătălin Frâncu, Nerdvana 2.0 București
- stud. Aureliu Valentin Antonie, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București
- prof. Bogdan-Ioan Popa, Liceul Teoretic Just 4Kids București
- stud. Alin Gabriel Răileanu, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Cristian Luchian, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Andrei Boacă, Facultatea de Informatică, Universitatea „Al. I. Cuza” Iași
- stud. Rareș-Andrei Cotoi, Facultatea de Matematică și Informatică, Universitatea „Babeș-Bolyai” Cluj-Napoca
- stud. Lucian-Andrei Badea, TU Delft
- stud. Ioan-Cristian Pop, Facultatea de Automatică și Calculatoare, Universitatea Națională de Știință și Tehnologie Politehnica București